Contents

Behavioral Interview Questions — Senior DevOps Engineer **1**
The STAR Framework 1
Category 1: Incident & Crisis Management 1
Category 2: Technical Leadership & Influence 1
Category 3: Complexity & Scale 2
Category 4: Collaboration & Conflict 2
Category 5: Strategy & Vision 3
10 Questions to Ask the Interviewer 3

Behavioral Interview Questions — Senior DevOps Engineer

The STAR Framework

Every behavioral answer should follow: - **S**ituation — context and background (1-2 sentences) - **T**ask — your specific responsibility (1 sentence) - **A**ction — what YOU specifically did, step by step (60% of answer) - **R**esult — measurable outcome + what you learned (1-2 sentences)

Keep each answer to 2-3 minutes. Practice out loud.

Category 1: Incident & Crisis Management

Q: Tell me about the most critical production incident you've handled.

Framework: SEV-1 incident → detection → IC role → mitigation → RCA → prevention

Sample story structure: - We had a payment processing outage affecting 100% of checkout traffic for 28 minutes - I was the on-call SRE and became IC - Opened incident channel, assigned roles, ran through diagnostics - Identified DB connection pool exhaustion from a new query pattern - Mitigated by rolling back deploy + increasing pool temporarily - Post-mortem identified missing index; added query analysis to CI - Result: MTTR reduced from 45 min to 18 min average; zero repeat incidents

---

Q: Describe a time you had to make a difficult decision under pressure with incomplete information.

Framework: Incident or outage → time pressure → two bad options → decision + reasoning → outcome

Key points to hit: - Acknowledge the incomplete info explicitly - Explain your decision framework (reversibility, blast radius, speed) - Show you communicated your uncertainty to stakeholders - Demonstrate learning regardless of outcome

---

Q: Tell me about a major outage you caused. What happened?

⚠ Every senior engineer has caused at least one outage. Saying "I haven't" is a red flag.

Template: - Be direct and own it without excuses - Focus heavily on what you learned and what systemic changes you made - Interviewers want blameless mentality: "the system allowed this to happen" - Good ending: "as a result, I implemented X which prevented Y from ever happening again"

Category 2: Technical Leadership & Influence

Q: Tell me about a time you improved a team's processes significantly.

Strong answer elements: - Quantify the before state (manual deploys took 2 hours, 3 rollbacks per month) - Describe what you proposed and why (data-driven, not opinion) - Address resistance: "the team was skeptical because X, I addressed that by Y" - Quantify after state (deploys take 8 minutes, zero failed deploys in 6 months)

Q: Describe a situation where you had to push back on a deadline or feature request for reliability/security reasons.

Template: - Business wanted feature launched immediately - I identified risk: “skipping review would expose customer PII” - I presented options with tradeoffs, not a flat “no” - Option A: ship now, accept risk, timeline for fix - Option B: delay 2 days, ship securely - Framed as shared decision, not gatekeeping - They chose B (or chose A and I documented the risk formally)

Q: Tell me about a time you mentored someone or helped a team member grow.

Framework: junior engineer → specific skill gap → how you coached → their improvement

Good elements: - Pair programming on a specific problem, not just “I taught them” - Show you adapted your style to their learning style - Demonstrate follow-through: you didn’t just explain once, you followed up - Result: they solved the next problem independently

---

Category 3: Complexity & Scale

Q: Tell me about the most complex infrastructure you’ve designed or operated.

Hit these points: - Scale numbers (pods, requests/sec, data volume, regions) - Which problems were hardest and why - Key architectural decisions and their tradeoffs - What you’d do differently with today’s knowledge

---

Q: Describe a situation where you had to manage multiple competing priorities.

Framework: competing urgent requests + finite team capacity

Key behaviors to show: - Explicit prioritization criteria (business impact, SLA breach, customer count) - Proactive communication to all stakeholders with expected timelines - Not just firefighting — you addressed root causes to reduce future fires - You delegated or escalated where appropriate

---

Q: Tell me about a time you introduced a new technology or tool to your organization.

Strong narrative: - Problem the existing tool had (specific pain points with data) - How you evaluated alternatives (POC, benchmarks, team fit) - How you got buy-in (demo, data, phased rollout) - How you trained the team (runbooks, workshops, pair sessions) - What happened after rollout (adoption, problems overcome)

Category 4: Collaboration & Conflict

Q: Tell me about a conflict with a developer team over reliability requirements.

Templates to avoid: “I was right and convinced them”

Better framing: - Different valid perspectives (dev: ship fast, SRE: don’t break prod) - You sought to understand their constraints first - Found a compromise: error budget policy, feature flags, canary releases - Built shared ownership of reliability metrics

---

Q: Describe a time you worked with a team in a different timezone or geography.

Good elements: - What asynchronous practices you established (written over verbal, docs, video updates) - How you handled handoffs for on-call or incident response - How you built trust without face-to-face interaction

---

Category 5: Strategy & Vision

Q: Where do you see the biggest DevOps challenges in the next 3 years?

> Strong answer pillars: 1. **Platform Engineering** — standardized internal developer platforms replacing bespoke tooling 2. **AI in DevOps** — LLM-powered runbook generation, anomaly detection, code review 3. **Supply chain security** — SLSA, SBOM, code provenance becoming standard requirements 4. **FinOps** — cloud cost optimization becoming a first-class engineering concern 5. **Developer Experience** — measuring and improving DORA metrics systematically

---

Q: How do you stay current in this fast-moving field?

> Name specific sources (be honest): - Blogs: Martin Fowler, Netflix Tech Blog, Uber Engineering, Cloudflare Blog - Podcasts: The Changelog, DevOps Handbook, SRE Weekly - Conferences: KubeCon, re:Invent, FOSDEM - Learning: Weekly dedicated time, certifications (CKA, AWS Solutions Architect) - Community: Local meetups, CNCF Slack, contributing to OSS

---

10 Questions to Ask the Interviewer

1. “How do you measure the health of your DevOps/platform team? What metrics matter most?”
2. “What does a typical on-call rotation look like? How many pages per week does the average engineer receive?”
3. “What’s the biggest reliability or operational challenge you’re facing right now?”
4. “How do developers and platform engineers collaborate on new service deployment requirements?”
5. “What’s the current deployment frequency, and where do you want it to be?”
6. “How are post-mortems conducted? Are they blameless in practice, not just in policy?”
7. “What does the cloud infrastructure look like today, and where are you trying to get to?”
8. “How much toil reduction opportunity exists in this role — what would success look like in year one?”
9. “What are the main technology bets the company is making over the next 12-18 months?”
10. “What’s the biggest thing that would make the person in this role unsuccessful?”